Corrigé - Initiation à Python

Chapitre 1 - Découvrir Python

Exercice 1 ⋆

	Type d'erreur	Cause
a	`SyntaxError`	guillemet fermant manquant
b	`NameError`	la variable `agee` n'existe pas
c	`TypeError`	`input` renvoie du texte, pas un nombre
d	`IndentationError`	le bloc du `if` n'est pas indenté
e	`ZeroDivisionError`	division par zéro

Les corrections :

```
print("Bonjour")                    # a : on ferme le guillemet

age = 15                            # b : on corrige la faute de frappe
print("Vous avez", age, "ans")

n = int(input("Un entier : "))      # c : on convertit en entier
print(n + 1)

if 3 > 2:                           # d : on indente le bloc
    print("Trois est plus grand")
```

Pour **e**, il n'y a pas de correction *syntaxique* : il faut décider ce que le programme doit faire lorsque `b` vaut 0, par exemple afficher un message.

À retenir. Les trois premières erreurs sont détectées à des moments différents : `SyntaxError` avant même l'exécution, les deux autres pendant. C'est pourquoi le programme **b** affiche parfois quelque chose avant de s'arrêter.

Exercice 2 ⋆

```
7
3 + 4
34
10
22222
Bonjour Marie
BonjourMarie
```

1. Sans guillemets, `3 + 4` est un *calcul* : Python affiche 7. Avec guillemets, `"3 + 4"` est un *texte* : Python le recopie tel quel.
2. Les deux résultats qui surprennent sont `"3" + "4"`, qui donne `34` et non 7, et `"2" * 5`, qui donne `22222` et non 10.

Sur les chaînes de caractères, le signe `+` ne signifie pas « additionner » mais « coller bout à bout », et `*` signifie « répéter ». Le même symbole a donc deux sens, selon le *type* de ce qui l'entoure. On notera aussi la différence entre la virgule et le `+` dans `print` : la virgule insère automatiquement une espace, pas le `+`.

Exercice 3 ⋆

1. Le programme affiche `55.08`, sans aucune indication de ce que ce nombre représente.

2. Une réécriture possible :

```
# Calcul du prix TTC d'un article a partir de son prix hors taxes

prix_ht = 45.9          # prix hors taxes, en euros
taux_tva = 0.2          # taux de TVA : 20 %

prix_ttc = prix_ht * (1 + taux_tva)

print("Prix TTC :", prix_ttc, "euros")
```

3. Trois commentaires suffisent : un pour dire ce que fait le programme, deux pour préciser l'unité et la signification des données.

Le commentaire `# on multiplie p par 1+t` est inutile.

Il ne fait que répéter en français ce que le code dit déjà. Un bon commentaire explique le *pourquoi*, pas le *comment* : ici, que $1+t$ correspond au prix augmenté de la TVA.

Exercice 4 ⋆⋆

1. Dans la console, chaque ligne saisie qui produit une valeur voit son résultat affiché automatiquement :

```
>>> 2 + 3
5
>>> x = 7
>>> x * 2
14
```

La ligne `x = 7` n'affiche rien : une affectation ne *produit* pas de valeur, elle range une valeur quelque part.

2. Exécuté comme script, le programme n'affiche *rien du tout*. Les calculs sont bien effectués, mais rien ne demande de les montrer.

Dans un script, seul `print` affiche quelque chose.

3.

```
print(2 + 3)
x = 7
print(x * 2)
```

4. La console convient pour tester une idée, vérifier une syntaxe ou faire un calcul ponctuel : la réponse est immédiate et rien n'est conservé. Le script convient dès qu'un programme doit être relu, corrigé, réexécuté ou transmis.

Chapitre 2 - Variables, types et calculs

Exercice 5 ⋆

Ligne	Instruction	a	b
1	`a = 5`	5	—
2	`b = 3`	5	3
3	`a = a + b`	8	3
4	`b = a - b`	8	5
5	`a = a - b`	3	5

Le programme affiche `3 5` : il a *échangé* le contenu des deux variables, sans utiliser de variable auxiliaire. L'astuce consiste à stocker temporairement la somme dans `a`.
Version plus courte, propre à Python :

```
a, b = b, a
```

Le piège. L'ordre des lignes 3 à 5 est impératif. Le débutant écrit souvent `a = b` puis `b = a`, ce qui perd la valeur initiale de `a` dès la première ligne : les deux variables valent alors 3.

Exercice 6 ⋆

1.

Nom	Valide ?	Raison
`prix`	oui	
`2eme_note`	**non**	commence par un chiffre
`note2`	oui	un chiffre est permis, mais pas en tête
`prix total`	**non**	contient une espace
`prix_total`	oui	
`print`	valide, mais à proscrire	masque la fonction `print`
`Prix`	oui	différent de `prix` : la casse compte
`if`	**non**	mot réservé du langage
`_x`	oui	le tiret bas est autorisé en tête

2. `moyenne` serait le meilleur nom. Parmi ceux proposés, aucun n'est explicite : `prix` et `note2` induiraient le lecteur en erreur.

3. Après `print = 5`, le nom `print` ne désigne plus la fonction d'affichage mais l'entier 5. L'instruction `print("Bonjour")` provoque alors :

```
TypeError: 'int' object is not callable
```

Python tente en effet d'« appeler » le nombre 5 comme une fonction. L'erreur est d'autant plus déroutante qu'elle survient sur une ligne parfaitement correcte.

Exercice 7 ⋆

Expression	Type	Valeur	Expression	Type	Valeur
`7 / 2`	`float`	3.5	`int("12") + 3`	`int`	15
`7 // 2`	`int`	3	`float("3.5") * 2`	`float`	7.0
`7 % 2`	`int`	1	`str(12) + "3"`	`str`	`"123"`
`7.0 // 2`	`float`	3.0	`int(7 / 2)`	`int`	3
`2 ** 10`	`int`	1024	`round(7 / 2)`	`int`	4

Les deux expressions de même valeur mais de types différents sont `7 // 2`, qui vaut 3 (entier), et `7.0 // 2`, qui vaut 3.0 (flottant). Dès qu'un flottant intervient, le résultat est flottant, même si la valeur est entière.

Deux pièges.

- `int(7 / 2)` vaut 3 et `round(7 / 2)` vaut 4 : `int` *tronque* vers zéro, `round` *arrondit*. Ils diffèrent dès que la partie décimale atteint 0,5.
- `str(12) + "3"` donne `"123"` et non `"15"` : une fois convertis en texte, les nombres se collent au lieu de s'ajouter.

Exercice 8 ⋆⋆

1. La division entière donne les heures, le reste donne les minutes :

```
minutes = int(input("Nombre de minutes : "))

heures = minutes // 60
reste = minutes % 60

print(heures, "h", reste, "min")
```

```
Nombre de minutes : 200
3 h 20 min
```

2. On applique deux fois le même principe : d'abord les heures, puis les minutes sur ce qui reste.

```
total = int(input("Nombre de secondes : "))

heures = total // 3600
reste = total % 3600
minutes = reste // 60
secondes = reste % 60

print(heures, "h", minutes, "min", secondes, "s")
```

3.

```
Nombre de secondes : 3725
1 h 2 min 5 s
```

Vérification : $3\,725 = 1 \times 3600 + 2 \times 60 + 5$ ✓

Exercice 9 ⋆⋆

```
from math import sqrt

xa = float(input("xA : "))
ya = float(input("yA : "))
xb = float(input("xB : "))
yb = float(input("yB : "))

distance = sqrt((xb - xa) ** 2 + (yb - ya) ** 2)

print("AB =", round(distance, 2))
print("Milieu :", (xa + xb) / 2, ";", (ya + yb) / 2)
```

```
xA : 1
yA : 2
xB : 4
yB : 6
AB = 5.0
Milieu : 2.5 ; 4.0
```

On utilise `float` et non `int` : les coordonnées peuvent être décimales. La fonction `round(x, 2)` arrondit à deux décimales.
Attention à ne pas écrire `sqrt(xb - xa ** 2 + ...)` : les parenthèses sont indispensables, car `**` est prioritaire sur la soustraction.

Exercice 10 ⋆⋆⋆

1. Une remise de 20 % revient à multiplier par 0,8, une remise de 10 % à multiplier par 0,9.

```
prix = float(input("Prix initial : "))

apres_1 = prix * 0.8
prix_final = apres_1 * 0.9

print("Prix final :", round(prix_final, 2))
```

2. Le client a tort. On ajoute le calcul de la remise globale :

```
remise = (1 - prix_final / prix) * 100
print("Remise globale :", round(remise, 2), "%")
```

```
Prix initial : 100
Prix final : 72.0
Remise globale : 28.0 %
```

3. Non, le pourcentage global ne dépend pas du prix initial : on obtient 28 % pour 100 euros comme pour 45,90 euros. En effet, le prix final vaut :

$$p \times 0{,}8 \times 0{,}9 = 0{,}72\,p$$

Le prix initial p se simplifie dans le calcul de la remise, qui vaut donc toujours $1 - 0{,}72 = 0{,}28$.

Deux remises de 20 % et 10 % équivalent à une seule remise de 28 %.

Les pourcentages de remise ne s'additionnent pas : ils se *composent*, par multiplication des coefficients.

4.

```
prix = float(input("Prix initial : "))
t1 = float(input("Premiere remise, en % : "))
t2 = float(input("Seconde remise, en % : "))

prix_final = prix * (1 - t1 / 100) * (1 - t2 / 100)
remise = (1 - prix_final / prix) * 100

print("Prix final :", round(prix_final, 2))
print("Remise globale :", round(remise, 2), "%")
```

Chapitre 3 - Tests et conditions

Exercice 11 ⋆

Expression	Valeur	Expression	Valeur
`a < b`	`True`	`not (a == b)`	`True`
`a == 5`	`True`	`a < b < 20`	`True`
`a != b`	`True`	`mot == "Python"`	`False`
`a > 3 and b > 15`	`False`	`len(mot) == 6`	`True`
`a > 3 or b > 15`	`True`	`a % 2 == 0`	`False`

Première surprise : `mot == "Python"` vaut `False`. Python distingue les majuscules des minuscules : la chaîne `"python"` n'est pas la chaîne `"Python"`. C'est une source d'erreur fréquente lorsqu'on compare une réponse saisie par l'utilisateur.
Seconde surprise : `a < b < 20` vaut `True`. Cette écriture, calquée sur la notation mathématique $a < b < 20$, signifie `a < b and b < 20`. Peu de langages l'autorisent, et l'on y gagne en lisibilité.
On notera au passage que `len(mot) == 6` vaut `True` : la fonction `len` compte les *caractères* d'une chaîne, et `"python"` en possède six.
Enfin `a % 2 == 0` teste la parité : ici 5 est impair, d'où `False`.

Exercice 12 ⋆

1.

```
x = float(input("Un nombre : "))

if x > 0:
    print("positif")
elif x < 0:
    print("negatif")
else:
    print("nul")
```

2.

```
n = int(input("Un entier : "))

if n % 2 == 0:
    print("pair")
else:
    print("impair")
```

3. On compare le premier aux deux autres, puis le deuxième au troisième :

```
a = float(input("a : "))
b = float(input("b : "))
c = float(input("c : "))

if a >= b and a >= c:
    plus_grand = a
elif b >= c:
    plus_grand = b
else:
    plus_grand = c

print("Le plus grand est", plus_grand)
```

Le `elif b >= c` n'a pas besoin de retester `b >= a` : si l'on y arrive, c'est que la première condition était fausse.

Exercice 13 ⋆⋆

1. Les quatre erreurs :

Ligne	Erreur	Message
1	`input` renvoie du texte	`TypeError` à la comparaison
2	deux-points manquants	`SyntaxError`
5	`print` non indenté	`IndentationError`
—	aucun contrôle de validité	pas d'erreur, mais un résultat faux

2.

```
note = float(input("Votre note : "))

if note >= 16:
    print("Mention tres bien")
elif note >= 14:
    print("Mention bien")
elif note >= 12:
    print("Mention assez bien")
else:
    print("Pas de mention")
```

3. Les notes 16, 14 et 12 donnent respectivement les trois mentions, et 8 n'en donne aucune. La note 20 affiche « Mention très bien », ce qui est correct ; mais -3 affiche « Pas de mention », sans signaler que la note est absurde.

4.

```
note = float(input("Votre note : "))

if note < 0 or note > 20:
    print("Note invalide")
elif note >= 16:
    print("Mention tres bien")
elif note >= 14:
    print("Mention bien")
elif note >= 12:
    print("Mention assez bien")
else:
    print("Pas de mention")
```

Le test de validité doit venir *en premier* : placé à la fin, il ne serait jamais atteint, puisque l'une des branches précédentes aurait déjà été exécutée.

Exercice 14 ⋆⋆

```
a = float(input("Premier cote : "))
b = float(input("Deuxieme cote : "))
c = float(input("Troisieme cote : "))

if a < b + c and b < a + c and c < a + b:
    print("Ces longueurs forment un triangle.")

    # triangle rectangle : reciproque de Pythagore
    if a**2 == b**2 + c**2 or b**2 == a**2 + c**2 or c**2 == a**2 + b**2:
        print("Il est rectangle.")

    # cas particuliers
    if a == b and b == c:
        print("Il est equilateral.")
    elif a == b or b == c or a == c:
        print("Il est isocele.")
else:
    print("Ces longueurs ne forment pas un triangle.")
```

Test	Réponse
$(3\,;4\,;5)$	triangle, rectangle
$(2\,;2\,;2)$	triangle, équilatéral
$(1\,;1\,;5)$	pas un triangle, car $5 > 1+1$
$(5\,;5\,;7)$	triangle, isocèle

Attention à l'ordre du dernier test : `elif` garantit qu'un triangle équilatéral n'est pas aussi annoncé comme isocèle. Avec deux `if` indépendants, les deux messages s'afficheraient.

Exercice 15 ⋆⋆⋆

1. 1900 est divisible par 100 mais pas par 400 : elle n'est pas bissextile. 2000 est divisible par 400 : elle l'est. 2023 n'est pas divisible par 4 : elle ne l'est pas. 2024 est divisible par 4 sans l'être par 100 : elle l'est.

2. On teste les cas du plus particulier au plus général :

```
annee = int(input("Une annee : "))

if annee % 400 == 0:
    print("bissextile")
elif annee % 100 == 0:
    print("non bissextile")
elif annee % 4 == 0:
```

```
        print("bissextile")
    else:
        print("non bissextile")
```

3. En un seul test :

```
    if (annee % 4 == 0 and annee % 100 != 0) or annee % 400 == 0:
        print("bissextile")
    else:
        print("non bissextile")
```

La condition traduit mot pour mot la règle : « divisible par 4 mais pas par 100 », *ou bien* « divisible par 400 ».

4. Les deux versions donnent les mêmes réponses sur 1900, 2000, 2023 et 2024, ainsi que sur toutes les années de 1890 à 1910. On peut le vérifier automatiquement :

```
    for annee in range(1890, 1911):
        v1 = version_1(annee)
        v2 = version_2(annee)
        if v1 != v2:
            print("Desaccord en", annee)
    print("Verification terminee")
```

Aucun désaccord n'apparaît — et l'on peut pousser la vérification de l'an 1 à l'an 3 000 sans trouver de contre-exemple. C'est une bonne habitude : plutôt que de comparer deux programmes à la main sur quelques cas, on fait comparer la machine sur des milliers.

Chapitre 4 - Les boucles

Exercice 16 ⋆

Boucle	**Affichage**
`range(5)`	0, 1, 2, 3, 4 (un par ligne)
`range(1, 5)`	1, 2, 3, 4
`range(0, 10, 3)`	0, 3, 6, 9
`range(5, 0, -1)`	5, 4, 3, 2, 1
`range(3)` avec `end=""`	`***` sur une seule ligne

Trois points à retenir. La borne de départ vaut 0 par défaut ; la borne d'arrivée est toujours *exclue* ; un pas négatif permet de compter à rebours, mais il faut alors que le départ soit supérieur à l'arrivée.

`range(a, b)` produit $b - a$ valeurs, de a à $b - 1$.

C'est la source d'erreur la plus fréquente : pour parcourir 1 à 10, il faut écrire `range(1, 11)`.

Exercice 17 ⋆

1.

```
    for i in range(1, 11):
        print(7, "x", i, "=", 7 * i)
```

2.

```
    table = int(input("Quelle table ? "))
    for i in range(1, 11):
        print(table, "x", i, "=", table * i)
```

3. On utilise un *accumulateur*, initialisé à 0 avant la boucle :

```
    somme = 0
    for i in range(1, 101):
        somme = somme + i
    print("La somme vaut", somme)
```

```
La somme vaut 5050
```

4. Pour un produit, l'accumulateur s'initialise à 1 et non à 0 :

```
produit = 1
for i in range(1, 11):
    produit = produit * i
print("10! =", produit)
```

```
10! = 3628800
```

Le piège. Initialiser le produit à 0 donnerait 0, puisque tout produit par zéro est nul. L'élément neutre dépend de l'opération : 0 pour une somme, 1 pour un produit.

Exercice 18 ⋆⋆

1. $u_1 = 2 \times 3 - 5 = 1$, puis $u_2 = 2 \times 1 - 5 = -3$, puis $u_3 = 2 \times (-3) - 5 = -11$.

2.

```
u = 3
for n in range(10):
    print("u", n, "=", u)
    u = 2 * u - 5
```

L'ordre est essentiel : on affiche u_n *avant* de calculer u_{n+1}, faute de quoi le premier terme n'apparaîtrait jamais.

3.

```
n = int(input("Quel rang ? "))
u = 3
for i in range(n):
    u = 2 * u - 5
print("u", n, "=", u)
```

4. Les dix premiers termes valent :

$$3\,;\ 1\,;\ -3\,;\ -11\,;\ -27\,;\ -59\,;\ -123\,;\ -251\,;\ -507\,;\ -1019$$

La suite est donc *décroissante*, et semble tendre vers $-\infty$.

Avec $u_0 = 5$, tous les termes valent 5 : la suite est constante, car $2 \times 5 - 5 = 5$. Le nombre 5 est un *point fixe* de la relation.

Avec $u_0 = 6$, la suite croît : 6, 7, 9, 13, 21, 37, ...

Le comportement dépend entièrement de la position de u_0 par rapport à 5.

Exercice 19 ⋆⋆

1. La boucle extérieure compte les lignes, l'intérieure les étoiles de chaque ligne :

```
for i in range(1, 6):
    for j in range(i):
        print("*", end="")
    print()          # passe a la ligne suivante
```

Le `print()` vide, placé dans la boucle extérieure, termine la ligne courante.

2.

```
n = int(input("Combien de lignes ? "))
for i in range(1, n + 1):
    print("*" * i)
```

On peut se passer de la boucle intérieure : `"*" * i` répète l'étoile i fois.

3. Il suffit de parcourir les lignes à rebours :

```
for i in range(5, 0, -1):
    print("*" * i)
```

4. On complète à gauche par des espaces, de sorte que chaque ligne ait la même largeur :

```
n = 5
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * i)
```

```
    *
   **
  ***
 ****
*****
```

Exercice 20 ⋆⋆

1.

```
n = 0
while 2 ** n <= 1000000:
    n = n + 1
print("Le plus petit n vaut", n, "et 2^n =", 2 ** n)
```

```
Le plus petit n vaut 20 et 2^n = 1048576
```

2.

```
capital = 1000
annees = 0
while capital <= 2000:
    capital = capital * 1.03
    annees = annees + 1
print("Il faut", annees, "annees")
print("Capital :", round(capital, 2), "euros")
```

```
Il faut 24 annees
Capital : 2032.79 euros
```

3. Non : une boucle `for` exige de connaître *à l'avance* le nombre de répétitions. Or c'est précisément ce nombre que l'on cherche.

On utilise `while` quand la condition d'arrêt porte sur le résultat, non sur un compteur.

4. C'est déjà fait dans le programme ci-dessus, grâce à `round(capital, 2)`.

Exercice 21 ⋆⋆

1. La variable `n` part de 10 et *augmente* à chaque tour. La condition `n > 0` reste donc vraie indéfiniment : la boucle ne s'arrête jamais.

Dans une boucle `while`, le corps doit faire progresser vers la condition d'arrêt.

2.

```
n = 10
while n > 0:
    print(n)
    n = n - 1
```

3. La variable `x` est divisée par 2 à chaque tour, à partir de 100. La boucle s'arrête dès que `x` descend à 1 ou en dessous :

$$100 \to 50 \to 25 \to 12{,}5 \to 6{,}25 \to 3{,}125 \to 1{,}5625 \to 0{,}78125$$

La boucle s'exécute 7 fois, et le programme affiche 7.

4. Avec `x > 0`, la boucle deviendrait presque infinie : en divisant par 2, on n'atteint jamais 0 exactement en mathématiques. En pratique, les flottants finissent par valoir exactement 0 après environ mille itérations, par limitation du codage machine — mais c'est un accident, pas une intention.

Exercice 22 ⋆⋆⋆

1.

```
n = int(input("Un entier superieur a 2 : "))
for d in range(1, n + 1):
    if n % d == 0:
        print(d, "divise", n)
```

2. On compte les diviseurs, ou l'on teste seulement ceux strictement compris entre 1 et n :

```
n = int(input("Un entier : "))
est_premier = True
for d in range(2, n):
    if n % d == 0:
        est_premier = False
print(est_premier)
```

3. Deux améliorations : s'arrêter dès qu'un diviseur est trouvé, et ne tester que jusqu'à $\sqrt{n}$.

```
n = int(input("Un entier : "))
est_premier = True
d = 2
while d * d <= n and est_premier:
    if n % d == 0:
        est_premier = False
    d = d + 1
print(est_premier)
```

Pourquoi $\sqrt{n}$ suffit. Si $n = a \times b$ avec $a \leqslant b$, alors $a^2 \leqslant ab = n$, donc $a \leqslant \sqrt{n}$. Tout diviseur non trivial va donc par paire avec un autre, et le plus petit des deux est inférieur à $\sqrt{n}$: s'il n'y a rien jusqu'à $\sqrt{n}$, il n'y a rien du tout.

4.

```
for n in range(2, 100):
    if est_premier(n):
        print(n, end=" ")
```

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
```

Il y en a 25.

5. Pour $n = 9\,973$, qui est premier :

Version	**Nombre de tests**
question 2, sans arrêt anticipé	9 971
question 3, jusqu'à $\sqrt{n}$	98

Environ 100 fois moins de tests, pour un résultat identique.

Chapitre 5 - Les fonctions

Exercice 23 ⋆

```
from math import pi

def aire_rectangle(L, l):
    return L * l

def perimetre_cercle(r):
    return 2 * pi * r

def celsius_vers_fahrenheit(c):
    return 9 * c / 5 + 32
```

Un programme qui les utilise :

```
longueur = float(input("Longueur : "))
largeur = float(input("Largeur : "))
print("Aire :", aire_rectangle(longueur, largeur))

temperature = float(input("Temperature en degres Celsius : "))
print("Soit", celsius_vers_fahrenheit(temperature), "degres Fahrenheit")
```

Vérifications : `aire_rectangle(3, 4)` vaut 12, `perimetre_cercle(1)` vaut environ 6,283, et `celsius_vers_fahrenheit(100)` vaut 212.

Exercice 24 ⋆

1. `print(double_a(5))` affiche 10. En revanche `print(double_b(5))` affiche *deux* lignes :

```
10
None
```

La fonction affiche 10 elle-même, puis `print` affiche ce qu'elle a renvoyé, c'est-à-dire `None`.

2. `double_a(5) + 1` vaut 11. Mais `double_b(5) + 1` provoque une erreur :

```
TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

3. Seule `double_a` est réutilisable, car elle *renvoie* une valeur, que l'on peut ensuite stocker, additionner ou passer à une autre fonction. La seconde se contente d'afficher : le résultat est perdu dès qu'il est écrit à l'écran.

> `return` transmet une valeur au programme ; `print` l'écrit à l'écran.

4. Une fonction sans `return` renvoie `None`, une valeur spéciale signifiant « rien ».

Exercice 25 ⋆⋆

Programme a.

```
3
10
```

L'affectation `x = 3` dans la fonction crée une variable *locale*, qui n'existe que pendant l'appel. Le `x` global, qui vaut 10, n'est pas touché.

Programme b.

```
5
```

Le paramètre `n` est une copie : le modifier ne change pas `a`. De plus, la valeur renvoyée par `g(a)` n'est ni stockée ni affichée : elle est perdue.

Programme c.

```
7
```

Ici aucune variable locale `y` n'est créée : Python va donc chercher `y` dans la portée *globale*, où elle vaut 7. Une fonction peut lire une variable globale, mais elle ne peut pas la modifier par une simple affectation.

> Une variable créée dans une fonction est locale : elle disparaît à la fin de l'appel.

Exercice 26 ⋆⋆

```
def est_pair(n):
    return n % 2 == 0

def somme_chiffres(n):
    somme = 0
    while n > 0:
        somme = somme + n % 10      # le chiffre des unites
        n = n // 10                 # on supprime ce chiffre
    return somme

def est_divisible_par_3(n):
    return somme_chiffres(n) % 3 == 0
```

Vérification : `somme_chiffres(1234)` vaut 10, puisque $1+2+3+4=10$.
On notera que `est_pair` s'écrit en une ligne : la comparaison `n % 2 == 0` produit déjà un booléen, il suffit de le renvoyer. Écrire `if ... : return True else: return False` serait redondant.

4.

```
for n in range(1, 51):
    if est_divisible_par_3(n) != (n % 3 == 0):
        print("Desaccord en", n)
print("Verification terminee")
```

Aucun désaccord : le critère de divisibilité par 3 est confirmé sur les cinquante premiers entiers.

Exercice 27 ⋆⋆⋆

1. L'algorithme d'Euclide remplace le couple $(a\,;b)$ par $(b\,;a \bmod b)$ jusqu'à ce que le reste soit nul :

```
def pgcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

2.

```
def ppcm(a, b):
    return a * b // pgcd(a, b)
```

On utilise `//` et non `/` : le quotient est exact, et l'on souhaite un entier.

3.

```
def simplifie(a, b):
    d = pgcd(a, b)
    return a // d, b // d
```

4.

```
>>> simplifie(252, 300)
(21, 25)
>>> simplifie(1001, 1430)
(7, 10)
```

Les PGCD valent respectivement 12 et 143.

5.

```
def sont_premiers_entre_eux(a, b):
    return pgcd(a, b) == 1
```

L'intérêt des fonctions apparaît ici. Les quatre dernières s'appuient toutes sur `pgcd`, écrite une seule fois. Si l'on découvrait une erreur dans l'algorithme d'Euclide, une seule correction suffirait à réparer l'ensemble.

Chapitre 6 - Les listes

Exercice 28 ⋆

1. `notes[0]` vaut 12, `notes[3]` vaut 6, et `notes[-1]` vaut 9 : l'indice -1 désigne le dernier élément.

2. `len(notes)` vaut 6, et le dernier indice est donc 5.

Les indices vont de 0 à `len(L)` $-\,1$.

3. `notes[6]` provoque une erreur :

```
IndexError: list index out of range
```

4. `notes[1:4]` vaut `[8, 15, 6]`, `notes[:3]` vaut `[12, 8, 15]`, et `notes[3:]` vaut `[6, 17, 9]`. La borne de gauche est incluse, celle de droite exclue — comme pour `range`.

5. Après `notes[1] = 14`, la liste vaut :

`[12, 14, 15, 6, 17, 9]`

Après `notes.append(11)`, elle vaut `[12, 14, 15, 6, 17, 9, 11]` et compte sept éléments.

Exercice 29 ⋆

1.

```
carres = []
for k in range(1, 21):
    carres.append(k * k)
print(carres)
```

2.

```
v = 1
suite = [v]
for n in range(14):
    v = 3 * v + 2
    suite.append(v)
print(suite)
```

Les premiers termes sont 1, 5, 17, 53, 161, 485, et le quinzième vaut 9 565 937.

3.

```
opposes = []
for x in L:
    opposes.append(-x)
```

4.

```
resultat = []
for n in range(1, 51):
    if n % 3 == 0 or n % 5 == 0:
        resultat.append(n)
print(resultat)
```

```
[3, 5, 6, 9, 10, 12, 15, 18, 20, 21, 24, 25,
 27, 30, 33, 35, 36, 39, 40, 42, 45, 48, 50]
```

Le schéma est toujours le même : une liste vide avant la boucle, un `append` à l'intérieur. C'est l'analogue, pour les listes, de l'accumulateur des sommes.

Exercice 30 ⋆⋆

1. Oui, les deux boucles affichent exactement la même chose.

2. Il faut la seconde, car le numéro de la note est l'*indice* :

```
for i in range(len(notes)):
    print("La note numero", i, "vaut", notes[i])
```

3. Il faut également la seconde : pour *modifier* la liste, il faut désigner la case à changer.

```
for i in range(len(notes)):
    notes[i] = notes[i] + 1
```

La boucle `for note in notes` ne conviendrait pas : `note` est une copie de la valeur, et écrire `note = note + 1` ne modifierait pas la liste.

4.

> Parcours par élément : quand on a seulement besoin des *valeurs*.
>
> Parcours par indice : quand on a besoin de la *position*, ou qu'on veut *modifier*.

Exercice 31 ⋆⋆

1.

```
[1, 2, 3, 4]
[1, 2, 3, 4]
```

L'instruction `b = a` ne crée pas une nouvelle liste : elle donne un *second nom* à la même liste. Modifier `b` revient donc à modifier `a`.

2. Il faut demander explicitement une copie :

```
b = a[:]          # ou bien : b = list(a)
```

On obtient alors `[1, 2, 3]` et `[1, 2, 3, 4]`.

3. Le second programme affiche 5 : aucun piège ici.

4. La différence tient à la nature des objets. Un entier est *immuable* : `y = y + 1` ne modifie pas l'ancien nombre, elle fabrique un nouveau nombre et redirige `y` vers lui, laissant `x` intact. Une liste est *modifiable* : `b.append(4)` transforme la liste elle-même, que tous ses noms voient changer.

> Le piège ne concerne que les objets modifiables : listes, et non nombres ou chaînes.

Exercice 32 ⋆⋆

```
def somme(L):
    s = 0
    for x in L:
        s = s + x
    return s

def longueur(L):
    n = 0
    for x in L:
        n = n + 1
    return n

def moyenne(L):
    return somme(L) / longueur(L)

def minimum(L):
    m = L[0]
    for x in L:
        if x < m:
            m = x
    return m

def compte(L, x):
    c = 0
    for element in L:
        if element == x:
            c = c + 1
    return c

def inverse(L):
    resultat = []
    for i in range(longueur(L) - 1, -1, -1):
        resultat.append(L[i])
    return resultat
```

Vérification sur `[3, 7, 3, 1, 9]` : somme 23, longueur 5, moyenne 4,6, minimum 1, `compte(L, 3)` vaut 2, et `inverse(L)` vaut `[9, 1, 3, 7, 3]`.

Deux remarques. Dans `minimum`, on initialise avec `L[0]` et non avec 0 : une liste peut ne contenir que des nombres négatifs. Et `inverse` construit une *nouvelle* liste, conformément à l'énoncé : la liste de départ n'est pas touchée.

Exercice 33 ⋆⋆⋆

1.

```
def effectifs(notes):
    eff = [0] * 21          # 21 cases, d'indice 0 a 20
    for n in notes:
        eff[n] = eff[n] + 1
    return eff
```

L'astuce consiste à utiliser la note elle-même comme *indice* de la case à incrémenter.

2.

```
def mediane(notes):
    triee = sorted(notes)
    n = len(triee)
    if n % 2 == 1:
        return triee[n // 2]
    else:
        return (triee[n // 2 - 1] + triee[n // 2]) / 2
```

3.

```
def au_dessus(notes):
    m = moyenne(notes)
    c = 0
    for n in notes:
        if n > m:
            c = c + 1
    return c
```

4. Sur la liste proposée :

```
Liste triee : [5, 6, 8, 9, 11, 12, 13, 14, 15, 17]
Moyenne : 11.0
Mediane : 11.5
Au-dessus de la moyenne : 5 eleves sur 10
```

5. Oui, c'est possible. Il suffit qu'une valeur très basse tire la moyenne vers le bas : avec `[0, 10, 10, 10, 10]`, la moyenne vaut 8 et *quatre* élèves sur cinq sont au-dessus.

La moyenne n'est pas la médiane : elle est sensible aux valeurs extrêmes.

C'est exactement ce que mesure la médiane, qui coupe l'effectif en deux parts égales par construction.

Chapitre 7 - Quelques algorithmes classiques

Exercice 34 ⋆⋆

1.

```
def maximum(L):
    m = L[0]
    for x in L:
        if x > m:
            m = x
    return m
```

2. On mémorise l'indice plutôt que la valeur :

```
def indice_maximum(L):
    imax = 0
    for i in range(len(L)):
        if L[i] > L[imax]:
            imax = i
    return imax
```

3. Le test étant `>` strict, l'indice ne change pas en cas d'égalité :

> La fonction renvoie l'indice de la *première* occurrence du maximum.

Avec `>=`, elle renverrait celui de la dernière. Ce choix doit être explicite : c'est exactement le genre de détail qui provoque des désaccords entre deux programmes pourtant tous deux « corrects ».

4. On maintient deux variables, mises à jour dans le bon ordre :

```
def deux_plus_grands(L):
    if L[0] >= L[1]:
        m1, m2 = L[0], L[1]
    else:
        m1, m2 = L[1], L[0]
    for x in L[2:]:
        if x > m1:
            m2 = m1        # l'ancien premier devient deuxieme
            m1 = x
        elif x > m2:
            m2 = x
    return m1, m2
```

Vérifications : sur `[3, 7, 3, 1, 9]` la fonction renvoie $(9\,;7)$, et sur `[5, 2, 8, 1, 9, 3]` elle renvoie $(9\,;8)$.
Le point délicat est la ligne `m2 = m1`, qui doit précéder `m1 = x` : dans l'ordre inverse, l'ancien maximum serait perdu.

Exercice 35 ⋆⋆

1. La fonction $f : x \mapsto x^3 + x - 1$ est une somme de fonctions croissantes sur $[0\,;1]$. De plus $f(0) = -1 < 0$ et $f(1) = 1 > 0$: la solution est donc unique et située dans $]0\,;1[$.

2.

```
def f(x):
    return x**3 + x - 1

x = 0
pas = 0.01
while f(x) < 0:
    x = x + pas

print("La solution est entre", round(x - pas, 4), "et", round(x, 4))
```

```
La solution est entre 0.68 et 0.69
```

3.

```
pas = float(input("Quel pas ? "))
```

4. La solution vaut environ 0,6823. Avec un pas de 10^{-6}, il faut parcourir l'intervalle depuis 0, soit environ :

$$\frac{0{,}6823}{10^{-6}} \approx 682\,300 \text{ itérations}$$

> Le balayage est simple, mais très coûteux dès que la précision augmente.

Exercice 36 ⋆⋆⋆

1. À chaque étape, on coupe l'intervalle en deux et l'on conserve la moitié qui contient la solution, repérée par un changement de signe :

```
def dichotomie(a, b, precision):
    while b - a > precision:
        m = (a + b) / 2
        if f(a) * f(m) <= 0:
            b = m            # la solution est dans [a ; m]
        else:
            a = m            # elle est dans [m ; b]
    return a, b
```

2.

```
>>> dichotomie(0, 1, 1e-6)
(0.6823273, 0.6823282)
```

Le programme effectue 20 étapes, contre 682 300 itérations pour le balayage.

La dichotomie est environ 34 000 fois plus rapide.

3.

```
def dichotomie(a, b, precision):
    etapes = 0
    while b - a > precision:
        m = (a + b) / 2
        if f(a) * f(m) <= 0:
            b = m
        else:
            a = m
        etapes = etapes + 1
    print("Nombre d'etapes :", etapes)
    return a, b
```

4. L'amplitude vaut 1 au départ et se divise par 2 à chaque étape : après n étapes, elle vaut $\dfrac{1}{2^n}$. On cherche donc le plus petit n tel que :

$$\frac{1}{2^n} < 10^{-6} \qquad \text{soit} \qquad 2^n > 10^6$$

L'exercice 20 a établi que le plus petit tel n vaut 20, ce que le programme confirme.
L'intérêt de la dichotomie tient à cette division par 2 : gagner une décimale supplémentaire coûte trois ou quatre étapes, alors que le balayage doit multiplier ses itérations par dix.

Exercice 37 ⋆⋆⋆

1. À chaque tour, on cherche le minimum de la partie non encore triée et on l'échange avec le premier élément de cette partie :

```
def tri_selection(L):
    for i in range(len(L) - 1):
        imin = i
        for j in range(i + 1, len(L)):
            if L[j] < L[imin]:
                imin = j
        L[i], L[imin] = L[imin], L[i]
    return L
```

2. En ajoutant `print(L)` après l'échange, on suit le déroulement sur `[5, 2, 8, 1, 9, 3]` :

```
[1, 2, 8, 5, 9, 3]
[1, 2, 8, 5, 9, 3]
[1, 2, 3, 5, 9, 8]
[1, 2, 3, 5, 9, 8]
[1, 2, 3, 5, 8, 9]
```

On observe que certains tours n'échangent rien : c'est le cas lorsque le minimum est déjà à sa place, l'échange se faisant alors d'un élément avec lui-même.

3. Le premier tour effectue $n-1$ comparaisons, le deuxième $n-2$, et ainsi de suite :

$$(n-1) + (n-2) + \cdots + 2 + 1 = \frac{n(n-1)}{2}$$

n	**Comparaisons mesurées**	$\dfrac{n(n-1)}{2}$
5	10	10
10	45	45
20	190	190

Ce nombre ne dépend pas de la liste : le tri par sélection effectue toujours le même nombre de comparaisons, que la liste soit déjà triée ou non.

4. Il suffit d'inverser le test, en cherchant le maximum au lieu du minimum :

```
if L[j] > L[imax]:
```

Exercice 38 ⋆⋆

1.

```
from random import randint

n = 1000
effectifs = [0] * 7          # on ignore la case 0

for lancer in range(n):
    face = randint(1, 6)
    effectifs[face] = effectifs[face] + 1

for face in range(1, 7):
    print("Face", face, ":", effectifs[face] / n)
```

2. Non, les six fréquences ne sont pas égales. Sur un essai, on obtient par exemple :

$$0{,}160 \quad 0{,}166 \quad 0{,}165 \quad 0{,}163 \quad 0{,}183 \quad 0{,}163$$

Elles varient d'un essai à l'autre, et tournent autour de $\frac{1}{6} \approx 0{,}167$.

3. Avec 100 000 lancers, les fréquences se resserrent nettement :

$$0{,}1664 \quad 0{,}1673 \quad 0{,}1653 \quad 0{,}1685 \quad 0{,}1670 \quad 0{,}1654$$

C'est la loi des grands nombres : les fréquences se rapprochent des probabilités.

4.

```
from random import randint

n = 100000
succes = 0

for essai in range(n):
    if randint(1, 6) + randint(1, 6) == 7:
        succes = succes + 1

print("Frequence :", succes / n)
```

On obtient environ 0,166, à comparer à $\frac{1}{6} \approx 0{,}1667$. La somme 7 est en effet la plus probable : elle s'obtient de six façons sur les trente-six lancers possibles.

Problèmes

Exercice 39 ⋆⋆⋆

1.

```
from random import randint

secret = randint(1, 100)
proposition = int(input("Votre proposition : "))

while proposition != secret:
    if proposition > secret:
        print("Trop grand")
    else:
        print("Trop petit")
```

```
        proposition = int(input("Votre proposition : "))

    print("Gagne !")
```

La première lecture doit avoir lieu *avant* la boucle, sinon la condition porterait sur une variable inexistante. La seconde, à la fin du corps, prépare le tour suivant.

2.

```
coups = 1
while proposition != secret:
    ...
    proposition = int(input("Votre proposition : "))
    coups = coups + 1

print("Gagne en", coups, "coups !")
```

3. On ajoute une seconde condition d'arrêt :

```
while proposition != secret and coups < 7:
    ...

if proposition == secret:
    print("Gagne en", coups, "coups !")
else:
    print("Perdu ! Le nombre etait", secret)
```

4. L'ordinateur doit couper l'intervalle en deux à chaque coup :

```
a = 1
b = 100
reponse = ""

while reponse != "=":
    milieu = (a + b) // 2
    print("Est-ce", milieu, "?")
    reponse = input("Tapez +, - ou = : ")
    if reponse == "+":
        a = milieu + 1        # le nombre est plus grand
    elif reponse == "-":
        b = milieu - 1        # il est plus petit

print("Trouve :", milieu)
```

Attention aux `+ 1` et `- 1` : sans eux, le milieu déjà testé resterait dans l'intervalle et la boucle pourrait ne jamais s'arrêter.

5. À chaque question, le nombre de candidats est divisé par deux. En partant de 100 candidats :

$$100 \to 50 \to 25 \to 13 \to 7 \to 4 \to 2 \to 1$$

Il faut donc au plus sept questions. On peut aussi raisonner par les puissances de 2 :

$$2^6 = 64 < 100 \leqslant 128 = 2^7$$

Sept questions suffisent, et six ne suffiraient pas toujours.

Un programme testant les cent valeurs possibles confirme que le pire cas vaut exactement sept coups.

Exercice 40 ⋆⋆⋆

1. `ord("a")` vaut 97, et `chr(100)` vaut `"d"`. Les vingt-six minuscules occupent les codes 97 à 122, dans l'ordre alphabétique.

2. On se ramène à un rang entre 0 et 25, on décale, puis on revient à une lettre :

```
def decale(lettre, k):
    if "a" <= lettre <= "z":
        rang = ord(lettre) - ord("a")      # entre 0 et 25
```

```
            nouveau = (rang + k) % 26          # le modulo referme l'alphabet
            return chr(nouveau + ord("a"))
        return lettre
```

C'est le `% 26` qui fait revenir au début : avec $k = 1$, la lettre `z` a pour rang 25, et $(25+1) \bmod 26 = 0$, c'est-à-dire `a`.

3.

```
def chiffre(message, k):
    resultat = ""
    for lettre in message:
        resultat = resultat + decale(lettre, k)
    return resultat
```

Les espaces et la ponctuation traversent `decale` sans être modifiés, grâce au `return lettre` final.

4. Déchiffrer revient à décaler dans l'autre sens :

```
def dechiffre(message, k):
    return chiffre(message, -k)
```

Le `% 26` accepte les valeurs négatives et referme l'alphabet de la même façon.

5.

```
>>> dechiffre("dpvu tvs mf uspof", 1)
'cout sur le trone'
```

6. Il n'y a que vingt-cinq clés possibles : on peut toutes les essayer.

```
message = "yhql ylgl ylfl"
for k in range(1, 26):
    print(k, ":", dechiffre(message, k))
```

La clé $k = 3$ donne un texte qui a un sens :

```
3 : veni vidi vici
```

Le chiffre de César ne résiste pas : vingt-cinq essais suffisent à le casser.

C'est la faiblesse de fond de cette méthode : la clé appartient à un ensemble minuscule, que l'on parcourt entièrement en une fraction de seconde.

Exercice 41 ⋆⋆⋆⋆

1. En partant de 6 :

$$6 \to 3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$$

2.

```
def syracuse(n):
    termes = [n]
    while n != 1:
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
        termes.append(n)
    return termes
```

On utilise `//` et non `/` : la division exacte produirait des flottants, et le test `n != 1` finirait par comparer 1.0 à 1.

3.

```
def temps_de_vol(n):
    return len(syracuse(n)) - 1

def altitude_max(n):
    return max(syracuse(n))
```

On retranche 1 car la liste contient u_0 en plus des étapes effectuées. Pour $n = 6$, le temps de vol vaut 8 et l'altitude maximale 16.

4.

```
meilleur = 1
for n in range(1, 1001):
    if temps_de_vol(n) > temps_de_vol(meilleur):
        meilleur = n
print(meilleur, "avec un temps de vol de", temps_de_vol(meilleur))
```

```
871 avec un temps de vol de 178
```

5. Le même programme, avec `altitude_max`, donne :

```
703 avec une altitude de 250504
```

Non, ce n'est pas le même entier : 871 pour le temps, 703 pour l'altitude.

Un vol peut donc être long sans monter très haut, et inversement. On notera l'ampleur du second nombre : parti de 703, la suite atteint plus de 250 000 avant de redescendre.

6. Le nuage de points est d'une irrégularité frappante : deux entiers voisins peuvent avoir des temps de vol très différents — 27 demande 111 étapes, alors que 28 n'en demande que 18. On distingue toutefois des alignements de points, car les entiers d'une même « famille » finissent par rejoindre la même trajectoire.

Une conjecture ouverte. Personne n'a démontré que la suite atteint toujours 1. Le programme le vérifie pour tous les entiers testés, mais vérifier n'est pas démontrer : c'est la conjecture de Syracuse, ouverte depuis les années 1930.

Exercice 42 ⋆⋆⋆⋆

1. Le carré a pour aire 1, et le quart de disque de rayon 1 a pour aire $\dfrac{\pi \times 1^2}{4} = \dfrac{\pi}{4}$. Un point tiré au hasard dans le carré tombe donc dans le quart de disque avec la probabilité :

$$\frac{\pi/4}{1} = \frac{\pi}{4} \approx 0{,}785$$

2. On en déduit $\pi \approx 4 \times \dfrac{\text{points dans le disque}}{\text{points tirés}}$:

```
from random import random

n = int(input("Combien de points ? "))
dedans = 0

for tirage in range(n):
    x = random()
    y = random()
    if x * x + y * y <= 1:
        dedans = dedans + 1

print("Estimation de pi :", 4 * dedans / n)
```

3.

n	**Estimation**	**Décimales exactes**
100	3,000000	aucune
10 000	3,136800	une
1 000 000	3,140964	deux

Multiplier le nombre de points par 100 ne fait gagner qu'*une* décimale.

4. Non, trois exécutions avec $n = 10\,000$ donnent par exemple 3,1292, 3,1608 et 3,1568. Le résultat change à chaque fois, puisque les tirages sont aléatoires.

Une méthode probabiliste ne donne pas un résultat, mais une estimation.

5. Pour gagner une décimale, il faut environ multiplier n par 100 : obtenir π à 10^{-3} près demanderait de l'ordre de cent millions de tirages, sans aucune garantie.

La méthode est très inefficace pour calculer π.

La méthode des rectangles du cours atteint la même précision en quelques milliers d'itérations, et surtout elle est *déterministe* : elle donne toujours la même réponse, et l'on sait majorer l'erreur. L'intérêt de Monte-Carlo est ailleurs : elle s'applique encore quand le domaine est trop compliqué pour être découpé en rectangles.

Exercice 43 ⋆⋆⋆⋆

1. On barre les multiples de 2, puis ceux de 3, puis ceux de 5 — inutile d'aller plus loin, car $7 \times 7 = 49 > 30$. Il reste :

$$2\,;\,3\,;\,5\,;\,7\,;\,11\,;\,13\,;\,17\,;\,19\,;\,23\,;\,29$$

2.

```
def crible(N):
    est_premier = [True] * (N + 1)
    est_premier[0] = False
    est_premier[1] = False

    p = 2
    while p * p <= N:
        if est_premier[p]:
            multiple = p * p          # inutile de commencer avant
            while multiple <= N:
                est_premier[multiple] = False
                multiple = multiple + p
        p = p + 1

    resultat = []
    for n in range(2, N + 1):
        if est_premier[n]:
            resultat.append(n)
    return resultat
```

Deux optimisations : on s'arrête à $\sqrt{N}$, et l'on commence à barrer à p^2, car les multiples plus petits ont déjà été barrés par un facteur premier inférieur.

3. `len(crible(100))` vaut 25, et `len(crible(1000))` vaut 168.

4. Pour $N = 10\,000$, le crible est environ *neuf fois* plus rapide que le test de primalité appliqué à chaque entier.

Le crible traite tous les entiers d'un coup ; le test les traite un par un.

C'est une différence de nature : le crible ne teste jamais si un nombre est premier, il se contente d'éliminer ceux qui ne le sont pas.

5.

```
def goldbach(n):
    premiers = crible(n)
    for p in premiers:
        if (n - p) in premiers:
            return p, n - p
    return None
```

En parcourant les premiers dans l'ordre croissant, on obtient le couple dont le premier terme est le plus petit possible.

6.

```
for n in range(4, 1001, 2):
    if goldbach(n) is None:
        print("Contre-exemple :", n)
print("Verification terminee")
```

Aucun contre-exemple : la conjecture est vérifiée jusqu'à 1 000. Le couple dont le premier terme est le plus grand s'obtient pour :

$$\boxed{992 = 73 + 919}$$

Pour tous les autres entiers pairs testés, un premier plus petit que 73 convient.

Une seconde conjecture ouverte. Comme pour Syracuse, la vérification par ordinateur — poussée aujourd'hui bien au-delà de 10^{18} — ne constitue pas une démonstration. La conjecture de Goldbach, énoncée en 1742, reste ouverte.